Perl for Programmers
QUICK REFERENCE GUIDE

BASICS
Running Perl
Variables & sigils
Strings
Numbers & operators
CONTROL FLOW
Conditionals
Loops
DATA
Arrays
Hashes
References
FUNCTIONS
Subroutines
Scope
ADVANCED
Regular expressions
File I/O
OOP
Error handling
Modules
Gotchas

LANGUAGE REFERENCE

Perl

A pragmatic, high-level scripting language known for powerful text processing, flexible syntax, and the CPAN ecosystem. If you know Python, Ruby, or C — most of Perl will feel familiar with a handful of deliberate quirks.

`perl 5.x` `interpreted` `dynamically typed` `multi-paradigm`

01

Running Perl

basics

Every script starts with a *shebang* pointing to the Perl interpreter. The two essential pragmas — `strict` and `warnings` — should be in every file you write; they catch most beginner mistakes at compile time.

MY_SCRIPT.PL

```
#!/usr/bin/env perl
use strict;       # require variable declarations, ban barewords
use warnings;     # warn about suspicious constructs
use feature 'say';  # enable say(), given/when, etc.

say "Hello, world!";
```

WAYS TO RUN

```
perl script.pl              # run a script file
perl -e 'say "hi"'          # execute a one-liner
perl -c script.pl           # syntax-check only, don't run
perl -w script.pl           # enable warnings (prefer 'use warnings')
./script.pl                 # execute directly (needs chmod +x and shebang)
perl -d script.pl           # interactive debugger
```

USEFUL ONE-LINER FLAGS

FLAG	EFFECT	EXAMPLE
`-e`	execute a string as code	`perl -e 'print 42'`
`-n`	loop over input lines (sets `$_`), no print	`perl -ne 'print if /foo/'`
`-p`	same as -n but prints `$_` each iteration	`perl -pe 's/foo/bar/g'`
`-i`	edit files in-place (add extension for backup)	`perl -pi.bak -e 's/a/b/g' *.txt`
`-a`	auto-split `$_` into `@F` on whitespace	`perl -ane 'print $F[2]'`
`-l`	auto-chomp input; append `\n` to print	`perl -lne 'print uc'`

02

Variables & sigils

core concept

Perl's most distinctive feature is its **sigil system** — every variable is prefixed with a symbol that tells you its type. Unlike most languages, the sigil can change depending on how you access the variable.

```
my $scalar  = "one value";   # $ — a single value (string, number, ref)
my @array   = (1, 2, 3);     # @ — an ordered list of scalars
my %hash    = (   => 1);      # % — key/value pairs (dictionary/map)
my $ref     = \@array;        # $ — a reference is always a scalar
```

> KEY RULE — SIGIL REFLECTS WHAT YOU EXTRACT, NOT THE CONTAINER
> When you take *one element* out of an array or hash, the sigil becomes `$` (scalar). When you take *multiple elements* (a slice), the sigil becomes `@`.

```
$array[0]         # one element → scalar → $
@array[1,3]       # slice (multiple elements) → list → @
$hash{key}        # one hash value → scalar → $
@hash{qw(a b)}    # hash slice → list → @
```

DECLARATION — ALWAYS USE `MY`

```
my $x         # declare a lexical (block-scoped) variable
my ($a, $b)   # declare multiple at once
my @list      # declare an array
my %map       # declare a hash

# 'our' for package globals, 'local' for dynamic scope override
our $VERSION = "1.0";
local $/     = undef;     # temp override, restored on block exit
```

03

Strings

basics

QUOTING

```
'single quotes'         # literal — no interpolation, no escapes (except \' \\)
"double quotes"         # interpolates $vars and @arrays, processes \n \t etc.
q(same as single)       # q() = ''. Any delimiter works: q|..| q{..} q/..
qq(same as double)      # qq() = "". Any delimiter: qq|..| qq{..}
qw(word1 word2 word3)   # quote-words → ('word1','word2','word3') — great for lists
```

OPERATORS AND KEY FUNCTIONS

```
"hello" . " world"      # concatenation (. not +)
"ha" x 3               # repetition → "hahaha"
("x","y") x 2          # list repetition → ("x","y","x","y")

length($s)              # character count
substr($s, 2, 4)       # substr(string, offset, length)
index($s, "foo")       # first occurrence position (-1 if not found)
uc($s)  lc($s)          # uppercase / lowercase
chomp($s)               # remove trailing newline (modifies in place)
chop($s)                # remove and return last character
sprintf("%.2f", $n)    # formatted string (like printf to a variable)
split(/,/, $s)         # split string on regex → array
join(",", @a)           # join array elements into string
```

HEREDOC

```
my $text = <<END;         # interpolates (double-quote behaviour)
  Hello, $name.
END

my $raw = <<'END';       # quoted label → no interpolation
  Literal $text here.
END

my $ind = <<~END;        # ~ strips leading whitespace (5.26+)
    Can be indented.
    END
```

04

Numbers & operators

basics

Perl has no separate integer and float types — it converts automatically. String-to-number coercion is silent: `"42abc"` becomes `42` in numeric context.

ARITHMETIC

```
42   3.14   6.02e23   0xFF   0b1010   0777   1_000_000
+  -  *  /  %  **         # ** is exponentiation (no ^)
++  --                    # auto-increment/decrement (works on strings too!)
abs($n)   int($n)   sqrt($n)
```

COMPARISON — TWO COMPLETE SETS

> CRITICAL DIFFERENCE FROM OTHER LANGUAGES
> Perl has entirely separate operators for numeric and string comparison. Using `==` on strings silently converts them to numbers. `"foo" == "bar"` is *true* (both convert to 0).

NUMERIC	STRING	MEANING
`== !=`	eq ne	equal / not equal
`< >`	lt gt	less / greater than
`<= >=`	le ge	less/greater or equal
`<=>`	cmp	spaceship: returns -1, 0, or 1

LOGICAL OPERATORS — TWO SYNTAXES

```
# Symbol forms — HIGH precedence (use inside expressions)
$a && $b    $a || $b    !$a

# Word forms — LOW precedence (use at statement level)
$a and $b   $a or $b    not $a

# Defined-or (5.10+) — preferred over || when 0 is a valid value
$val // "default"      # use right side only if left is undef
$val //= "default"     # assign default if $val is undef

# Classic idioms
open(my $fh, '<', $f) or die $!;    # or-die — open file or crash
```

TRUTHINESS

The following values are **false**; everything else is true:

```
undef   0   ""   "0"   ()   # ← "0" being false surprises people

# These are all TRUE:
"00"   "0.0"   "false"   0.0  (note: 0.0 == 0, so actually FALSE)
```

05

Conditionals

control flow

PYTHON / JS

```
if x > 0:
    print("pos")
elif x == 0:
    print("zero")
else:
    print("neg")
```

PERL

```
if ($x > 0) {
    say "pos";
} elsif ($x == 0) {
    say "zero";
} else {
    say "neg";
}
```

> NOTE
> It is `elsif`, not `else if` or `elif`. Curly braces are always required — no braceless one-liners like C.

UNLESS — "IF NOT"

```
unless ($done) { do_work(); }   # equivalent to: if (!$done) { ... }
```

POSTFIX (STATEMENT MODIFIER) FORM

Perl lets you put a single-statement condition *after* the action. This reads naturally and is idiomatic for guard clauses.

```
print "yes\n"  if     $flag;
print "no\n"   unless $flag;
return         if     !defined $input;   # guard clause
die "bad\n"    if     $error;
```

TERNARY OPERATOR

```
my $label = $n > 0 ? "positive" : "non-positive";

# Cascading ternary (format in columns for readability)
my $grade =   $s >= 90 ? 'A'
            : $s >= 80 ? 'B'
            : $s >= 70 ? 'C'
            :            'F';
```

06

Loops

control flow

```
# while / until
while ($i < 10)    { $i++ }
until ($done)      { work() }     # loops while condition is FALSE

# do...while (always executes body at least once)
do {
    $input = <     >;
} while ($input !~ /^quit/);

# C-style for
for (my $i = 0; $i < 10; $i++) { say $i }

# foreach — iterate over a list
foreach my $item (@list) { say $item }
for my $item (@list)     { say $item }     # 'for' and 'foreach' are identical

# Default variable $_ — many operations use it implicitly
for (@list)          { print }          # $_ is each element; print prints $_
print "$_\n"          for @list;        # postfix form
```

LOOP CONTROL

```
last;           # break — exit the loop
next;           # continue — skip to next iteration
redo;           # restart current iteration without re-testing condition

# Labels for nested loop control
OUTER: for my $i (1..5) {
    for my $j (1..5) {
        next OUTER if $j == 3;   # skip outer iteration
        last OUTER if $i == 4;   # exit both loops
    }
}
```

07

Arrays

data structures

```
my @a = (1, 2, 3);          # declaration
my @b = qw(           );    # from whitespace-separated words
my @c = (1..10);            # range operator → (1,2,3,...,10)

$a[0]    $a[-1]            # first / last element
$#a                         # last index (= scalar(@a) - 1)
scalar @a                   # number of elements (in scalar context: $n = @a)

push    @a, "x";           # append to end
my $v = pop     @a;       # remove and return last
unshift @a, "x";           # prepend to front
my $v = shift   @a;       # remove and return first
splice(@a, $off, $len, @new);  # insert/remove at any position

sort              @a        # alphabetical
sort { $a <=> $b } @a     # numeric ascending ($a,$b are special sort vars)
sort { $b <=> $a } @a     # numeric descending
reverse @a                  # reversed list (returns new list)

grep { $_ > 5 } @a         # filter — returns elements where block is true
map  { $_ * 2 } @a         # transform — applies block to each, returns new list
map  { $_ => 1 } @a        # build a lookup hash from an array
```

08

Hashes

data structures

Hashes are Perl's associative arrays (Python dict / JS object). Keys are always strings; values are scalars.

```
my %h = (
          => "Alice",   # => is "fat comma" — auto-quotes left side
          => 30,
          => "Perl",
);

$h{    }                # access a value (bare word key is fine)
$h{"any string"}        # quoted key for special characters
$h{       }             # returns undef if key doesn't exist

keys   %h               # list of all keys (arbitrary order)
values %h               # list of all values
each   %h               # next (key, value) pair — use in while loop

exists  $h{   }         # true if key exists (even if value is undef)
defined $h{   }         # true if value is defined
delete  $h{   }         # remove key/value pair

# Iteration
for my $k (sort keys %h) {
    say "$k = $h{$k}";
}
```

09

References

pointers

A reference is a scalar that holds the memory address of another value — like a pointer in C or any object variable in Python/Java. References are how you pass arrays/hashes without copying, and how you build nested data structures.

```
# Create references
my $aref = \@array;           # reference to existing array
my $href = \%hash;            # reference to existing hash
my $sref = \$scalar;         # reference to scalar
my $cref = \&mysub;          # reference to subroutine

# Anonymous constructors (create inline)
my $aref = [1, 2, 3];          # [ ] = anonymous arrayref
my $href = {      => "Alice"}; # { } = anonymous hashref
my $cref = sub { $_[0] * 2 }; # sub { } = anonymous sub / lambda

# Dereference with arrow notation (preferred)
$aref->[0]             # element 0 of arrayref
$href->{     }          # value for 'name' in hashref
$cref->(@args)          # call a coderef

# Adjacent brackets don't need the arrow
$aref->[0]{    }        # same as $aref->[0]->{key}

# Dereference the whole thing
@{$aref}                # as array
%{$href}                # as hash

ref($aref)              # "ARRAY" — check the reference type
ref($href)              # "HASH"
ref($obj)               # "ClassName" for blessed objects
```

NESTED STRUCTURES

```
# Array of hashrefs — the most common pattern
my @people = (
    {      => "Alice",      => 30},
    {      => "Bob",        => 25},
);
$people[0]{     }       # "Alice"

# Hash of arrayrefs
my %tags = (      => ["scripting", "text"]);
$tags{     }[0]         # "scripting"

# Push into a nested structure
push @{$tags{     }}, "regex";
```

10

Subroutines

functions

All arguments arrive as a flat list in `@_`. There are no declared parameter lists — you unpack `@_` yourself. The return value is the last evaluated expression, or use an explicit `return`.

PYTHON

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Alice")
greet("Bob", greeting="Hi")
```

PERL

```
sub greet {
    my ($name, $greeting) = @_;
    $greeting //= "Hello";
    return "$greeting, $name!";
}

greet("Alice");
greet("Bob", "Hi");
```

NAMED PARAMETERS (COMMON IDIOM)

```
sub create_user {
    my (%args) = @_;
    my $name  = $args{     }  // "Guest";
    my $email = $args{      } or die "email required";
    return {      => $name,       => $email};
}

create_user(      => "Alice",       => "a@b.com");
```

CONTEXT-SENSITIVE RETURN

```
sub flexible {
    return wantarray ? (1,2,3) : "one-two-three";
}
my @list  = flexible();    # list context  → (1, 2, 3)
my $str   = flexible();    # scalar context → "one-two-three"
```

11

Scope

variables

KEYWORD	TYPE	VISIBILITY	USE FOR
`my`	lexical	enclosing `{}` block	everything — default choice
`our`	package global	entire package / file	shared globals, `$VERSION`
`local`	dynamic	current call stack frame	temporarily override a global (e.g. `$/`)

```
my $x = "outer";
{
    my $x = "inner";   # shadows outer $x in this block
    say $x;             # "inner"
}
say $x;                 # "outer" — inner $x is gone

# local temporarily replaces a global, restores on block exit
our $sep = ",";
{
    local $sep = "|";   # $sep is "|" only here and in any subs called
}                        # $sep is "," again
```

CLOSURES

```
sub make_adder {
    my $n = shift;
    return sub { $_[0] + $n };   # captures $n from enclosing scope
}
my $add5  = make_adder(5);
my $add10 = make_adder(10);
$add5->(3);    # 8
$add10->(3);   # 13
```

12

Regular expressions

core strength

Regex is deeply integrated in Perl — not an afterthought. Operators are first-class syntax, and regex patterns can be stored in variables and composed.

```
# Matching — =~ binds a string to a regex operation
$s =~  /pattern/         # true if matches
$s !~  /pattern/         # true if does NOT match
if (/pattern/)           # implicit match against $_ (very common in loops)

# Capture groups
if ($date =~ /(\d{4})-(\d{2})-(\d{2})/) {
    my ($y, $m, $d) = ($1, $2, $3);      # $1..$9 capture groups
}
# Named captures (cleaner)
$date =~ /(?<year>\d{4})-(?<month>\d{2})/;
say $+{year};               # named capture via %+

# Global match — all occurrences
my @nums = ($s =~ /\d+/g);  # list of all matches

# Substitution
$s =~ s/old/new/;          # replace first match
$s =~ s/old/new/g;         # replace all
$s =~ s/old/new/gi;        # case-insensitive + global
my $new = $s =~ s/a/b/gr;  # /r = return copy, don't modify $s

# Transliteration (character-by-character swap)
$s =~ tr/a-z/A-Z/;        # uppercase all letters
my $count = ($s =~ tr/aeiou//);   # count vowels (no replacement = count)

# Store a pattern
my $pat = qr/\d{4}-\d{2}/;  # compiled regex object
$s =~ $pat;
```

KEY MODIFIERS

FLAG	EFFECT
`i`	case-insensitive matching
`g`	global — find all occurrences
`m`	multiline — `^` and `$` match each line boundary
`s`	`.` matches newline too
`x`	extended — whitespace and `#comments` ignored in pattern
`r`	non-destructive — return modified copy, leave original
`e`	replacement in `s///` is evaluated as Perl code

QUICK SYNTAX REFERENCE

SYNTAX	MEANING
`.`	any character except newline
`\d \D`	digit / non-digit
`\w \W`	word char [a-zA-Z0-9_] / non-word
`\s \S`	whitespace / non-whitespace
`^ $ \A \z`	start/end of line / start/end of string
`\b`	word boundary
`* + ? {n,m}`	greedy quantifiers; add `?` for non-greedy: `*? +?`
`(…)`	capture group → $1, $2 …
`(?:…)`	non-capturing group
`(?=…) (?!…)`	lookahead / negative lookahead
`(?<=…) (?<!…)`	lookbehind / negative lookbehind
`a|b`	alternation — a or b

13

File I/O

I/O

```
# Always use three-argument open
open(my $fh, '<',  'in.txt')  or die $!;  # read
open(my $fh, '>',  'out.txt') or die $!;  # write (truncates)
open(my $fh, '>>', 'log.txt') or die $!;  # append
open(my $fh, '<:utf8', $path) or die $!;  # with encoding layer

# Read line by line
while (my $line = <$fh>) {
    chomp $line;
    # process $line
}

# Slurp entire file into a string
my $content = do { local $/; <$fh> };

# Read all lines into an array
my @lines = <$fh>;
chomp @lines;

# Write
print {$fh} "line\n";    # braces around filehandle — avoid ambiguity
say   {$fh} "line";

close $fh;

# Diamond operator — reads ARGV files or STDIN (great for filters)
while (<>) { print }

# File test operators
-e $path   # exists         -f  is plain file
-d $path   # is directory    -r  readable
-s $path   # file size       -M  age in days (last modified)
```

14

Object-oriented programming

objects

Perl OOP is built on three primitives: a **package** is a class, a **blessed reference** is an object, and any sub in the package is a method. It's manual but transparent.

```
package       ;
use strict; use warnings;

# Constructor — just a sub named 'new' by convention
sub new {
    my ($class, %args) = @_;
    return bless {                       # bless ties data to class
              => $args{    },
              => $args{     } // "...",
    }, $class;
}

# Accessor (getter/setter)
sub name {
    my $self = shift;
    $self->{     } = shift if @_;    # set if arg given
    return $self->{     };
}

sub speak {
    my $self = shift;
    printf "%s says %s\n", $self->{     }, $self->{      };
}

# Subclass
package Dog;
use parent 'Animal';    # inherit from Animal

sub new {
    my ($class, %args) = @_;
    $args{sound} //= "Woof";
    return $class->SUPER::new(%args);   # call parent constructor
}

# Usage
package main;
my $d = Dog->new(name => "Rex");
$d->speak();                             # "Rex says Woof"
ref($d);                                 # "Dog"
$d->isa('Animal');                      # 1 (true)
$d->can('speak');                      # returns coderef or undef
```

MODERN ALTERNATIVE

For serious OOP, use `Moose` or the lighter `Moo` from CPAN. They provide attribute declarations, type constraints, roles (mixins), and method modifiers — eliminating most boilerplate.

15

Error handling

`exceptions`

Perl uses `die` / `eval` as throw/try-catch. `die` can throw a string or an object. The error lands in `$@` after an `eval` block.

PYTHON

```
try:
    risky()
except ValueError as e:
    print(f"caught: {e}")
finally:
    cleanup()
```

PERL

```
eval {
    risky();
};
if (my $e = $@) {
    warn "caught: $e";
}
cleanup();  # no finally; just put after
```

```
# Throw a string
die "something failed at line 42";

# Throw an object (structured exceptions)
die { code => 404, msg => "not found" };
die MyException->new(msg => "bad");

# Check type of exception
eval { risky() };
if (ref $@ eq 'HASH')          { say $@->{msg} }
elsif ($@)                      { die $@ }      # re-throw unknown errors

# Carp module — reports error from the caller's location
use Carp qw(carp croak confess);
croak   "bad input";       # like die but points to caller
carp    "suspicious";      # like warn but points to caller
confess "deep error";     # die + full stack trace
```

16

Modules & CPAN

`packages`

```
# Loading modules
use List::Util qw(sum max min first any all);   # import specific subs
use File::Path;                                   # import defaults
use Scalar::Util ();                             # load without importing
use strict;                                       # pragma — no symbol

# use vs require
use     Foo;   # compile-time: load + import + run BEGIN block
require Foo;   # runtime: load only, no automatic import

# Writing a module (Foo.pm)
package Foo;
use strict; use warnings;
use Exporter 'import';
our @EXPORT_OK = qw(my_func another_func);   # export on request

sub my_func { ... }

1;   # REQUIRED — module must return a true value
```

ESSENTIAL STANDARD LIBRARY MODULES

MODULE	PURPOSE
`List::Util`	`sum`, `max`, `min`, `first`, `any`, `all`, `reduce`
`Scalar::Util`	`looks_like_number`, `blessed`, `reftype`, `weaken`
`File::Path`	`make_path`, `remove_tree` — mkdir -p / rm -rf
`File::Basename`	`dirname`, `basename`
`Cwd`	`cwd`, `abs_path`
`POSIX`	`floor`, `ceil`, `strftime`
`Data::Dumper`	pretty-print any data structure for debugging
`Storable`	deep copy (`dclone`), serialize/deserialize
`JSON`	encode/decode JSON (CPAN — or JSON::XS for speed)
`DBI`	database interface — works with any RDBMS
`LWP::UserAgent`	HTTP client
`Getopt::Long`	full-featured command-line option parsing

INSTALLING CPAN MODULES

```
cpanm Module::Name              # cpanminus — recommended
perl -MCPAN -e 'install Foo'    # built-in CPAN shell
apt/brew install perl-Foo       # system package manager
```

17

Gotchas for programmers

`differences`

Things that surprise people coming from Python, JavaScript, Ruby, or C.

1 — "0" IS FALSE

The string `"0"` is false. `"00"`, `"0.0"`, and `"false"` are all true. This catches everyone.

2 — == VS EQ

`==` is numeric, `eq` is string. `"foo" == 0` is true because both convert to 0. Always use `eq` for string comparison.

3 — PRINT WITHOUT COMMA

It is `print STDERR "msg"` (no comma between handle and string) but `print {$fh} "msg"` (braces) for variables. Many people write `print STDERR, "msg"` accidentally — this prints nothing to STDERR and sends `"msg"` to STDOUT.

4 — ARRAYS FLATTEN IN LISTS

`my @combined = (@a, @b)` merges the arrays. To keep them separate, use references: `my @of_arrays = (\@a, \@b)`.

5 — CONTEXT CHANGES EVERYTHING

`my $n = @array` gives the count, not the array. `my ($first) = @array` gives the first element because of list context on the left. Context is the hardest Perl concept to internalize.

6 — SEMICOLONS ARE REQUIRED

Every statement ends with `;`. The only exception is the last statement inside a block before `}`, but always include it.

7 — CURLY BRACES ARE ALWAYS REQUIRED

Unlike C, `if ($x) do_thing();` is illegal. You must write `if ($x) { do_thing(); }` — or use the postfix form: `do_thing() if $x;`.

8 — $\ VS `SAY`

`print` does not add a newline. Use `say` (requires `use feature 'say'`) or append `\n` manually. Setting `$\` adds a suffix to every `print` output.

SPECIAL VARIABLES WORTH KNOWING IMMEDIATELY

VARIABLE	MEANING
`$_`	default variable — used implicitly by most string/list operations and loops
`@_`	subroutine arguments — always unpack this first thing in a sub
`$!`	system error message/number — check after failed syscalls
`$@`	exception from last `eval` block
`$?`	exit status of last `system()` or backtick command
`$/`	input record separator (default `\n`); set to `undef` to slurp
`$0`	name of the running script
`@ARGV`	command-line arguments
`%ENV`	environment variables